Faculty of Computing and AI
Cairo University
CS361: Artificial Intelligence

Project: Hnefatafl

Game Description

The Viking Chess or simply **Tafl** is a two-player asymmetric strategy game. The defending side comprises 12 soldiers and a king, who start the game in a cross formation in the center of the board. The defending white army, surrounding the king try to defend him until the king is able to reach any of the four corners, while a larger black attacking army attempts to capture him. The attackers comprise 24 soldiers positioned in four groups of 6 around the perimeter of the board. All pieces move like the Rook in chess and pieces are taken by sandwiching. Sandwiching means a piece is trapped horizontally or vertically between two attacking pieces. The playing grid layout is 9x9 or 11x11.

Game Setup

- Initially, the King is placed on the central square (the Throne)
- 12 Defenders are placed around the King
- 24 Attackers are placed in four groups at the edges of the board

How the game goes?

- **Movement of Pieces**:
    - All pieces (including the king) move any number of empty squares in a straight horizontal or vertical line ONLY (like a Rook in Chess)
    - They cannot move off the edge of the board.
    - They cannot pass through other pieces
    - A piece cannot share a space with another piece.
- **Capturing Opponents** (Custodial Capture): A piece is captured and removed from the board if it is "sandwiched" between **two** opposing pieces on opposite sides (horizontally or vertically). Other cases as below:
    - You can remove an opponent’s piece if you capture it between the king’s throne and your piece; or between a corner space and your piece.
    - The king is unarmed; which means he cannot assist in capturing opponents.
    - Pieces may travel through a sandwich without being captured, but they may not stop inside a sandwich.
- **King's Escape**: The defenders win if the King reaches any of the four corner squares.

- **Capturing the KING**: The **attackers** can win in the following cases:
    - They surround the King on all four sides, preventing his movement.
    - If the king is against a wall, they can win by surrounding the other three sides.
    - If the king is against a corner, they can win by surrounding the other two sides.
- **Turn Order**: In Hnefatafl, the attacking player always moves first (one attacker per turn). After that, the turn goes to the defender (one defender per turn), and so on.

Game Resources

- You can check this video for more information
    - https://www.youtube.com/watch?v=fZ9cMj2Qn5Y
    - https://www.youtube.com/watch?v=pByySTArLe8
- You can try the game online:
    - https://playabstractgames.com/en/hnefatafl/

Instructions

- Students must work in **groups of 3 - 5** for their project. Students have to be from the **same lab** or from another lab taught by the same TA.
- The game should be implemented to be played in **Human vs. Computer** mode. The game that doesn't include this playing mode will not be graded.
- Students should utilize the **alpha-beta pruning** algorithm to decide on computer moves. No other algorithms allowed in this project.
- It is allowed to use **Prolog** or **Python** only [**1 Bonus grade** in case you choose Prolog].
- Each project is graded based on the availability of:
    - A **game controller** that organizes the game by switching roles between the two players, receives the user's move, updates the game board, and declares the "End of Game".
    - Suitable knowledge **representation** of the game state.
    - Adequate **utility function** that evaluates current game state with respect to a given player.
    - **Alpha-beta pruning** algorithm implementation. (You are allowed to use the draft implementation that was illustrated in your lab)
    - Support for different **difficulty levels** (Easy, Medium, Hard) each characterized through the depth of the algorithm. E.g.: Easy 1, Medium 3, Hard 5.
- **BONUS:** User interface in any language (Java, C#, Python, etc.)
- Projects submitted will be checked against each other and against possible implementations on the web an AI tools. Similar projects WILL NOT BE DISCUSSED.
- **Cheating Policy: Negative the project grade.**

Submission Details

You are required to submit **one zip file** containing the following:

- Your **code** (either **.pl** or **.py** file(s))
- A **text file** containing the team members' **names, IDs and groups** and a **link to a video** uploaded on your drive.

The zip file should follow the following naming convention: **ID1_ID2_ID3_Group**

In the video, each team member should speak (stating his/her ID at the start) and take one of the main parts in the code (representation, controller, alpha-beta, moves and utility) and explain it. The video should also include running the game as a demo showing how players take turns and how the computer automatically plays. The overall duration of the video should be around 10 minutes and shouldn't exceed 15 minutes.

Grading Criteria

Board and state representation	**1.5 marks**
Updating and printing board after each move	**1 mark**
Player switching (human vs. computer)	**0.5 marks**
Representing possible moves	**1.5 marks**
Utility function	**1.5 marks**
Difficulty levels (easy, medium, hard)	**1 mark**
Alpha-beta pruning algorithm	**1 mark**
Using Prolog	**1 Bonus mark**
GUI	**1 Bonus mark**

Possible Penalties

Not including the group and IDs in the file's naming convention	**The assignment won't be marked**
Not submitting the video required in deliverables section	**The assignment won't be marked**
Failing to implement a two-player mode	**-1 mark**
Using another algorithm like minimax	**-0.5 mark**
Team members are not from the same lab or not with the same TA	**-1 mark**
Team members are less than 3 or more than 5	**-1 mark**